

Spring Framework — From Java OOP to Enterprise Development

A complete, beginner-to-confident study guide

Module 1: Introduction to the Spring Ecosystem

Written for a learner who knows Java & OOP, and is starting Spring today.

0. Prerequisites — What You Already Know

Before we start, let's be clear about what you bring to the table. You know:

- How to write Java classes, objects, constructors
- Interfaces, inheritance, polymorphism, encapsulation, abstraction
- Collections (List, Map, Set) and exceptions
- How to import packages and use basic Java annotations (like `@Override`)

That is **enough**. Everything in Spring is built **on top of** these ideas. Spring does not replace Java — it organizes and automates the way you already build objects and connect them together.

REMEMBER

Spring is not a new language. It is a set of tools that manage the objects you already know how to build.

Throughout this guide, every new term is explained the moment it appears. If you ever see a word you don't recognize, keep reading — it will be defined within the next sentence or two.

1. What Is Spring?

Simple Explanation

Spring is a **framework** for building Java applications. Let's unpack "framework" first, because that word gets thrown around loosely.

- A **library** is a collection of code you call. *You* are in charge. You decide when to call `Collections.sort()`, and your program keeps control the whole time.
- A **framework** is different: *it* is in charge. You write pieces of code (classes, methods) and *plug them into* the framework. The framework decides when to call your code, in what order, and with what data.

Spring is a framework that takes care of **creating your objects, connecting them together, and managing them for the entire lifetime of your application**, so that you can focus on writing the logic that actually matters for your business (e.g., "calculate a student's grade" or "process a payment") instead of writing plumbing code ("create this object, then create that object, then wire them together, then remember to clean them up").

Why It Exists

Every non-trivial Java application is really just a big web of objects that depend on each other. A `Car` needs an `Engine`. An `OrderService` needs a `PaymentService` and a `Repository`. Someone has to:

1. Create all these objects in the right order
2. Give each object the other objects it depends on
3. Manage their lifetime (when to create them, when to destroy them)
4. Apply cross-cutting concerns like transactions, security, and logging

Spring's job is to do all four of these things for you, in a predictable, configurable, testable way.

Problem Without It

Without Spring (or something like it), you write this wiring code by hand, all over your application, in every class that needs a collaborator object. As an application grows, this becomes messy, repetitive, and hard to change — we will see exactly why in Section 7 and 8.

Java Example — the world without Spring

```
public class ReportService {
    public void generate() {
        // I am personally responsible for creating everything I need
        Database db = new Database("prod-connection-string");
        Formatter formatter = new PdfFormatter();
        Mailer mailer = new SmtMailer("smtp.company.com");

        // ... now use db, formatter, mailer
    }
}
```

`ReportService` is doing two jobs: its *actual* job (generating a report) and the *plumbing* job (constructing a `Database`, a `Formatter`, a `Mailer`). This mixing of responsibilities is exactly what Spring removes.

Spring Example — the same idea, Spring style

```

@Service
public class ReportService {

    private final Database db;
    private final Formatter formatter;
    private final Mailer mailer;

    // Spring builds Database, Formatter, and Mailer,
    // and simply *hands* them to ReportService here.
    public ReportService(Database db, Formatter formatter, Mailer mailer) {
        this.db = db;
        this.formatter = formatter;
        this.mailer = mailer;
    }

    public void generate() {
        // Only the actual job is left here
    }
}

```

Don't worry about `@Service` yet — we will explain it fully in Section 12. For now, just notice: `ReportService` no longer creates its own dependencies. Something *external* does that and *gives* them to it. That single shift is the heart of everything Spring does, and we will spend a lot of this guide making that idea completely natural to you.

What Happens Internally (preview)

At a high level, when your Spring application starts:

```

Your Application Starts
|
v
Spring reads your classes
|
v
Spring figures out which objects it needs to create
  (these are called "Beans" – Section 9)
|
v
Spring creates them in the correct dependency order
|
v
Spring connects them together (Dependency Injection)
|
v
Your application runs, using fully-wired objects

```

We will walk through this exact flow in much more technical detail in Section 15.

Real-Life Analogy

REAL LIFE: Imagine a restaurant. As a customer, you don't walk into the kitchen, chop the vegetables, cook the food, and plate it yourself. You simply say "I'd like the pasta," and the *restaurant system* (kitchen staff, chef, waiter) handles everything and delivers a finished plate to you.

JAVA: Without Spring, your class is like a customer who has to personally go into the kitchen and cook their own meal — i.e., manually `new` up every object it needs.

SPRING: Spring is the restaurant. You describe *what* you need (a `Database`, a `Mailer`), and Spring's "kitchen" (the container, Section 10) prepares it and delivers it to you, fully ready to use.

INTERVIEW TIP

If asked "What is Spring?", do **not** just say "a Java framework." Say: "Spring is a framework that manages the creation and wiring of Java objects for us, following the principle of Inversion of Control, so that application code can focus on business logic instead of object plumbing." This shows you understand the *why*, not just the label.

INTERVIEW QUESTIONS

Q (Beginner): What is Spring Framework? - *Short answer:* Spring is a framework for building Java applications that manages object creation and dependencies for you. - *Strong answer:* Spring is a lightweight, open-source Java framework built around the principle of Inversion of Control. It manages the lifecycle and wiring of application objects (called beans), and provides supporting modules for things like web applications (Spring MVC), data access (Spring Data), security (Spring Security), and more — all built on a consistent core programming model. - *Detailed explanation:* The key word is “manages.” Spring doesn’t just offer utility functions; it takes over responsibility for creating objects, injecting their dependencies, and controlling their lifecycle, freeing your code from that plumbing.

Q (Beginner): Is Spring a library or a framework, and what’s the difference? - A library is code *you* call, on *your* schedule. A framework calls *your* code, on *its* schedule (this is Inversion of Control, which we cover deeply in Section 7). Spring is a framework because it controls when your objects are created and how they are wired together.

CAN YOU TEACH THIS?

Try explaining, out loud or in writing: “*What is Spring, and how is it different from a library, to a friend who only knows plain Java?*”

A strong answer should include: - The distinction between “you call code” (library) vs. “code calls you” (framework) - That Spring specifically manages object creation and wiring - A simple analogy (restaurant, socket, etc.) - One line on *why* this matters (less plumbing code, more focus on business logic)

2. Why Spring Was Created

The World Before Spring: Java EE / J2EE (early 2000s)

Before Spring existed (Spring 1.0 arrived in 2003), enterprise Java development mostly meant **J2EE** (Java 2 Platform, Enterprise Edition — later renamed Jakarta EE, see Section 6). J2EE’s flagship technology for business logic was **EJB (Enterprise JavaBeans)**.

EJB tried to solve real problems (transactions, remote calls, security) but the way it solved them created new, painful problems:

- **Heavy, verbose code.** To create a simple business object, you often had to write multiple interfaces, a “home” interface, deployment descriptors in XML, and implement lifecycle callback methods you didn’t care about.
- **Tight coupling to the container.** Your business classes had to extend EJB base classes and implement EJB-specific interfaces. This meant your “business logic” was mixed with framework plumbing, and you basically **could not unit test your code outside an application server**. You needed a full server just to run a test.
- **Slow development cycle.** Every small change meant redeploying to a heavyweight application server.
- **XML overload.** Huge, error-prone deployment descriptors just to configure simple things.

The Problem, Concretely

Imagine writing a simple service to calculate a discount. In classic EJB-style J2EE, you were writing a lot of code whose entire purpose was to satisfy the *container’s* requirements — not to solve the actual business problem. Testing that logic meant starting an application server. This slowed teams down enormously and made Java enterprise development feel bureaucratic compared to simpler platforms.

Enter Spring — Rod Johnson’s Idea (2002-2003)

Rod Johnson, in his book *“Expert One-on-One J2EE Design and Development”* (2002), argued that you could build enterprise-grade applications using **plain old Java objects (POJOs)** — ordinary classes with no special base classes or container interfaces — plus a lightweight container that wires them together. This became Spring Framework.

The core promise of Spring was:

“You write plain Java classes. Spring handles the enterprise plumbing (wiring, transactions, etc.) *around* your classes, not *inside* them.”

IMPORTANT

This is why Spring is often called a **“non-invasive” or “lightweight” framework**: your business classes don’t need to extend Spring base classes or implement Spring interfaces to get Spring’s benefits. Compare this to old EJB, which was highly invasive.

What Problem Did Spring Actually Solve?

Problem in pre-Spring Java EE	How Spring Solved It
Business classes tightly coupled to container-specific base classes/interfaces	Spring works with plain Java objects (POJOs) — no mandatory base classes
Hard to unit test without a full application server	POJOs can be tested with plain JUnit, no server needed
Manual, repetitive object wiring code scattered everywhere	Centralized Dependency Injection managed by the Spring Container
Verbose, error-prone XML deployment descriptors for simple things	Simpler XML, then annotations, then Java-based configuration (Section 3)
Cross-cutting concerns (logging, transactions) mixed into business code	Aspect-Oriented Programming (AOP) — Spring Core module
Inconsistent data access code across different technologies	Consistent templates and abstractions (Spring Data access)

Real-Life Analogy

REAL LIFE: Imagine every employee at a company had to personally negotiate their own desk, chair, laptop, badge, and parking spot with five different departments before they could start working. That’s exhausting and distracts from their actual job.

JAVA (pre-Spring): Your business class had to personally deal with container lifecycle methods, lookups, and configuration before it could do its actual job.

SPRING: Spring acts like a well-run HR/Facilities department: it handles onboarding (creating and configuring objects) so the “employee” (your class) can just start working.

INTERVIEW TIP

A common interview question is “*Why was Spring created?*” The strongest answer names the specific pain point: **testability and coupling to heavyweight EJB containers** — not just a vague “Java EE was complicated.”

INTERVIEW QUESTIONS

Q (Intermediate): What problems did Spring solve compared to traditional EJB-based J2EE? - Strong answer: Spring let developers write plain Java objects instead of container-specific classes, made unit testing possible without a running server, replaced verbose configuration with simpler mechanisms, and separated cross-cutting concerns (transactions, logging, security) from business logic using AOP.

3. Evolution of Spring

We only cover history that explains *why modern Spring looks the way it does*. Here is the beginner-friendly timeline:

```
2002 Rod Johnson's book criticizes heavy EJB, proposes POJO-based design
2003 Spring Framework 1.0 released – XML configuration era begins
2006 Spring 2.x – XML gets namespaces, easier to write, still XML-heavy
2009 Spring 2.5 / 3.0 – Annotation-based configuration arrives
    (@Component, @Autowired, @Service, ...)
2009 Spring 3.0 – Java-based configuration arrives (@Configuration, @Bean)
2014 Spring Boot 1.0 – convention-over-configuration, auto-configuration,
    embedded servers, "just run main()"
2017+ Spring 5 – reactive programming support (WebFlux), Java 8+ baseline
2022 Spring 6 / Spring Boot 3 – moved to Jakarta EE 9+ namespace
    (javax.* -> jakarta.*), Java 17 baseline
Today Spring Boot is the default way almost everyone starts new Spring apps
```

Stage 1 — XML Configuration Era (2003+)

In early Spring, you configured your beans (objects Spring manages — Section 9) in XML files:

```
<bean id="engine" class="com.example.PetrolEngine" />
<bean id="car" class="com.example.Car">
  <constructor-arg ref="engine" />
</bean>
```

This was already a huge improvement over EJB because your `Car` and `PetrolEngine` classes were plain Java — no special interfaces required. But the XML itself grew large and repetitive for big applications.

Stage 2 — Annotation-Based Configuration (~2009, Spring 2.5/3.0)

Instead of describing beans in XML, you *mark* your classes directly with annotations, and Spring scans your codebase to find them:

```
@Component
public class PetrolEngine implements Engine { }

@Component
public class Car {
  private final Engine engine;
  public Car(Engine engine) { this.engine = engine; }
}
```

This is much closer to what you'll write day-to-day today, and we explain it in full depth in Sections 8–12.

Stage 3 — Java-Based Configuration (Spring 3.0+)

For cases where you still want centralized, explicit configuration (e.g., configuring a third-party class you don't own), Spring lets you write configuration **in Java itself**, instead of XML:


```

@Configuration
public class AppConfig {

    @Bean
    public Engine engine() {
        return new PetrolEngine();
    }

    @Bean
    public Car car(Engine engine) {
        return new Car(engine);
    }
}

```

This is type-safe (the compiler checks it) and refactor-friendly (rename a class, and your IDE updates the configuration too) — both big advantages over XML.

Stage 4 — Spring Boot (2014+)

Even with annotations and Java config, you still had to manually configure a lot of “boilerplate” infrastructure: web servers, dependency versions, common defaults. **Spring Boot** was created to solve *that* problem — not by replacing Spring Framework, but by sitting on top of it and providing:

- **Auto-configuration** — Boot looks at what is on your classpath and configures sensible defaults automatically.
- **Starter dependencies** — one dependency (e.g., `spring-boot-starter-web`) pulls in everything commonly needed for a task, with compatible versions.
- **Embedded servers** — no need to install and configure a separate Tomcat server; it’s bundled in your application.
- **Production-ready features** — health checks, metrics, etc., out of the box.

IMPORTANT — SPRING FRAMEWORK ≠ SPRING BOOT

This is one of the most common beginner confusions, so let’s be extremely precise:

- **Spring Framework** is the underlying engine: IoC container, Dependency Injection, AOP, MVC, etc. It provides the *concepts and machinery*.
- **Spring Boot** is a layer *on top of* Spring Framework that makes it fast and easy to *start* and *configure* a Spring application, using auto-configuration and starters, instead of manual setup.

Analogy: Spring Framework is like a car’s engine and all its core components. Spring Boot is the “push-button ignition and pre-set dashboard” that lets you drive off immediately without manually wiring the engine, fuel lines, and electronics yourself. The engine (Spring Framework) is still doing the real work underneath.

Technically accurate version: Spring Boot does not replace or reimplement Dependency Injection, the ApplicationContext, or AOP — it *configures Spring Framework for you automatically*, based on convention and the libraries present on your classpath. Every Spring Boot application is, underneath, a Spring Framework application.

Where We Are Today

Almost all new Spring projects start with **Spring Boot**, because it removes tedious setup. But the concepts Boot is automatically wiring — IoC, DI, Beans, ApplicationContext, AOP — are all **Spring Framework** concepts. This guide teaches you those underlying concepts first (labeled `SPRING FRAMEWORK CONCEPT`), then shows you the Boot convenience layered on top (labeled `SPRING BOOT CONVENIENCE`), so you understand what’s really happening instead of relying on “Boot magic.”

INTERVIEW QUESTIONS

Q (Beginner): What’s the difference between Spring and Spring Boot? - Strong answer: Spring Framework provides the core capabilities (IoC container, DI, AOP, MVC, data access abstractions). Spring Boot is built on top of Spring Framework and provides auto-configuration, starter dependencies, and embedded servers so you can get a Spring application running quickly with minimal manual configuration. Spring Boot doesn’t replace Spring Framework — it configures it for you.

Q (Intermediate): Why did XML configuration lose popularity? - Strong answer: XML was verbose, error-prone (no compile-time checking), and disconnected from the code it configured (renaming a class wouldn't update the XML automatically). Annotations and Java-based configuration are more concise, type-safe, and refactor-friendly.

COMMON MISTAKE

Saying "Spring Boot is a replacement for Spring." It is not a replacement — it's a convenience layer. Every Spring Boot app is still a full Spring Framework app underneath.
